Homework 07

All analyses use R. Confusion matrices have observed classes in rows and predicted classes in columns.

Question 1: LDA

(a) Class priors, means, and pooled covariance

```
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd

classes <- sort(unique(y))
n <- nrow(X)
p <- ncol(X)
K <- length(classes)

prior <- sapply(classes, function(k) mean(y == k))
names(prior) <- classes
mu <- t(sapply(classes, function(k) {
  colMeans(X[y == k, , drop = FALSE])
}))
rownames(mu) <- classes

Sigma <- matrix(0, p, p)
for (j in seq_along(classes)) {
  Z <- sweep(X[y == classes[j], , drop = FALSE], 2, mu[j, ], "-")
  Sigma <- Sigma + crossprod(Z)
}
Sigma <- Sigma / (n - K)
dimnames(Sigma) <- list(colnames(X), colnames(X))

prior
```

```
        0         1
0.6536797 0.3463203
```

```
knitr::kable(mu, digits = 4, format = "latex",
             booktabs = TRUE, longtable = FALSE, caption = "Class means")
```

```
knitr::kable(Sigma, digits = 4, caption = "Pooled within-class covariance")
```

Table 1: Class means

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
0	135.4603	2.6347	4.3442	23.9691	1.3179	52.3675	25.7375	15.9314	38.8543
1	143.7375	5.5249	5.4879	28.1202	1.6000	54.4938	26.6229	19.1453	50.2938

Table 2: Pooled within-class covariance

	sbp	tobacco	ldl	adiposity	famhist	typea	obesity	alcohol	age
sbp	405.4348	14.5852	4.5812	49.1644	0.3370	-15.5877	18.9388	64.4004	95.1339
tobacco	14.5852	19.2426	0.7632	7.5381	0.0158	-2.0573	1.8335	20.5170	22.7654
ldl	4.5812	0.7632	4.0006	6.0327	0.0919	0.3446	2.6601	-2.5329	6.4789
adiposity	49.1644	7.5381	6.0327	56.7529	0.4328	-5.3096	22.7079	16.1190	60.5087
famhist	0.3370	0.0158	0.0919	0.4328	0.2258	0.0811	0.1840	0.7685	0.9974
typea	-15.5877	-2.0573	0.3446	-5.3096	0.0811	95.5656	2.6401	7.9600	-20.2783
obesity	18.9388	1.8335	2.6601	22.7079	0.1840	2.6401	17.6154	4.6894	15.6970
alcohol	64.4004	20.5170	-2.5329	16.1190	0.7685	7.9600	4.6894	598.2766	27.8859
age	95.1339	22.7654	6.4789	60.5087	0.9974	-20.2783	15.6970	27.8859	184.1321

We estimate the priors by n_k/n and use each class's sample mean. The pooled covariance is

$$\hat{\Sigma} = \frac{1}{n-K} \sum_{k} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^\top.$$

Here, $n = 462$ and $K = 2$, so the denominator is 460.

(b) Manual LDA prediction

```
scores <- matrix(NA_real_, n, K)
for (j in seq_along(classes)) {
  w <- solve(Sigma, mu[j, ])
  scores[, j] <- as.vector(X %*% w) -
    0.5 * sum(mu[j, ] * w) + log(prior[j])
}
pred_manual <- classes[max.col(scores, ties.method = "first")]

data.frame(
  Object = c("X", "y", "mu", "Sigma", "scores"),
  Dimensions = c(paste(dim(X), collapse = " x "), length(y),
                 paste(dim(mu), collapse = " x "),
                 paste(dim(Sigma), collapse = " x "),
                 paste(dim(scores), collapse = " x "))
)
```

```
  Object Dimensions
1      X    462 x 9
2      y         462
3     mu       2 x 9
4  Sigma       9 x 9
5 scores    462 x 2
```

```r
cm_manual <- table(
  Observed = factor(y, levels = classes),
  Predicted = factor(pred_manual, levels = classes)
)
cm_manual
```

```
        Predicted
Observed   0   1
       0 258  44
       1  73  87
```

```r
mean(pred_manual != y)
```

```
[1] 0.2532468
```

Each observation is assigned to the class with the largest LDA score. The training error is **25.32%**; this is an in-sample error.

(c) Compare with MASS::lda

```r
fit_lda <- MASS::lda(
  x = X,
  grouping = factor(y, levels = classes),
  prior = as.numeric(prior)
)
pred_lda <- predict(fit_lda, X)$class
cm_lda <- table(
  Observed = factor(y, levels = classes),
  Predicted = factor(pred_lda, levels = classes)
)
cm_lda
```

```
        Predicted
Observed   0   1
       0 258  44
       1  73  87
```

```r
same_predictions <- all(as.character(pred_manual) == as.character(pred_lda))
same_predictions
```

```
[1] TRUE
```

The manual calculation and `MASS::lda` give identical predictions and confusion matrices.

Question 2: QDA and marginal screening

```r
zip_train <- read.csv(gzfile("data/zip-train.csv.gz"), header = FALSE)
zip_test <- read.csv(gzfile("data/zip-test.csv.gz"), header = FALSE)
ytr <- zip_train[[1]]
yte <- zip_test[[1]]
xtr <- as.matrix(zip_train[, -1])
xte <- as.matrix(zip_test[, -1])
```

```r
pixel_order <- order(apply(xtr, 2, var), decreasing = TRUE)
digits <- 0:9
run_qda <- function(m) {
  chosen <- pixel_order[seq_len(m)]
  fit <- MASS::qda(
    x = xtr[, chosen, drop = FALSE],
    grouping = factor(ytr, levels = digits)
  )
  pred <- predict(fit, xte[, chosen, drop = FALSE])$class
  list(
    confusion = table(
      Observed = factor(yte, levels = digits),
      Predicted = factor(pred, levels = digits)
    ),
    error = mean(as.character(pred) != as.character(yte))
  )
}
```

Pixels are ranked using training-set marginal variances, without standardization. The same selected pixels are retained in the test set.

(a) Forty predictors

```r
result40 <- run_qda(40)
result40$confusion
```

```
        Predicted
Observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
       5  12   0   3   5   1 131   0   0   6   2
       6   0   0   8   0   1   8 148   0   5   0
       7   1   0   6   1   6   0   0 124   2   7
       8   1   0  10  12   3   2   0   0 137   1
       9   0   0   1   2   7   1   1   4   9 152
```

```r
result40$error
```

```
[1] 0.1554559
```

The test misclassification rate is **15.55%**.

(b) Interpretation

QDA fits successfully: all ten within-class covariance matrices have full rank with 40 selected predictors. Using fewer predictors resolves the singularity problem here and makes covariance estimation easier. High marginal variance does not necessarily mean a pixel is useful for classification.

(c) Ten predictors

```
result10 <- run_qda(10)
result10$confusion
```

```
        Predicted
Observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
       2  19   2  95  14  14   3  22  21   5   3
       3   5   0   6 103   3  42   0   0   7   0
       4   1   5  14   0  95   2   4  15  17  47
       5  15   0   6  25   4  96   4   2   8   0
       6  41   0  15   4   0  18  78   7   5   2
       7   4   1   6   0   6   0   1 122   0   7
       8   1   2  11  36  12  11   4   1  74  14
       9   0   2   9   2  20   0   0  16  10 118
```

```
result10$error
```

```
[1] 0.3423019
```

The test error increases to **34.23%** with 10 predictors, compared with **15.55%** with 40. Using only 10 predictors loses useful information about digit shapes, outweighing the benefit of simpler covariance estimation.

Question 3: Compare LDA and QDA

We use seed **43207**, with **100 training and 1,000 test observations per class**. Both methods use all ten variables and equal class priors. The first two variables have means $(-1, 0)$ and $(1, 0)$, unit variances, and correlations ρ and $-\rho$. The remaining eight variables are independent standard normals.

```
set.seed(43207)
rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)

sample_data <- function(rho, n) {
  S <- diag(10)
  S[1, 2] <- S[2, 1] <- rho
  X1 <- MASS::mvrnorm(n, c(-1, rep(0, 9)), S)
  S[1, 2] <- S[2, 1] <- -rho
  X2 <- MASS::mvrnorm(n, c(1, rep(0, 9)), S)
  list(x = rbind(X1, X2), y = factor(rep(1:2, each = n)))
}

simulate <- function(rho) {
  train <- sample_data(rho, 100)
  test <- sample_data(rho, 1000)
  lda <- MASS::lda(train$x, train$y, prior = c(0.5, 0.5))
  qda <- MASS::qda(train$x, train$y, prior = c(0.5, 0.5))
  c(LDA = mean(predict(lda, test$x)$class != test$y),
    QDA = mean(predict(qda, test$x)$class != test$y))
}
```

(a) One dataset per rho

```
single <- t(sapply(rhos, simulate))
knitr::kable(data.frame(rho = rhos, single), digits = 4,
             caption = "Single-run test misclassification rates")
```

Table 3: Single-run test misclassification rates

rho	LDA	QDA
0.0	0.1650	0.1885
0.2	0.1825	0.2035
0.4	0.1825	0.2110
0.6	0.1720	0.1800
0.8	0.1710	0.1415
0.9	0.1770	0.1000

Within each setting, the two classifiers use the same training and test observations.

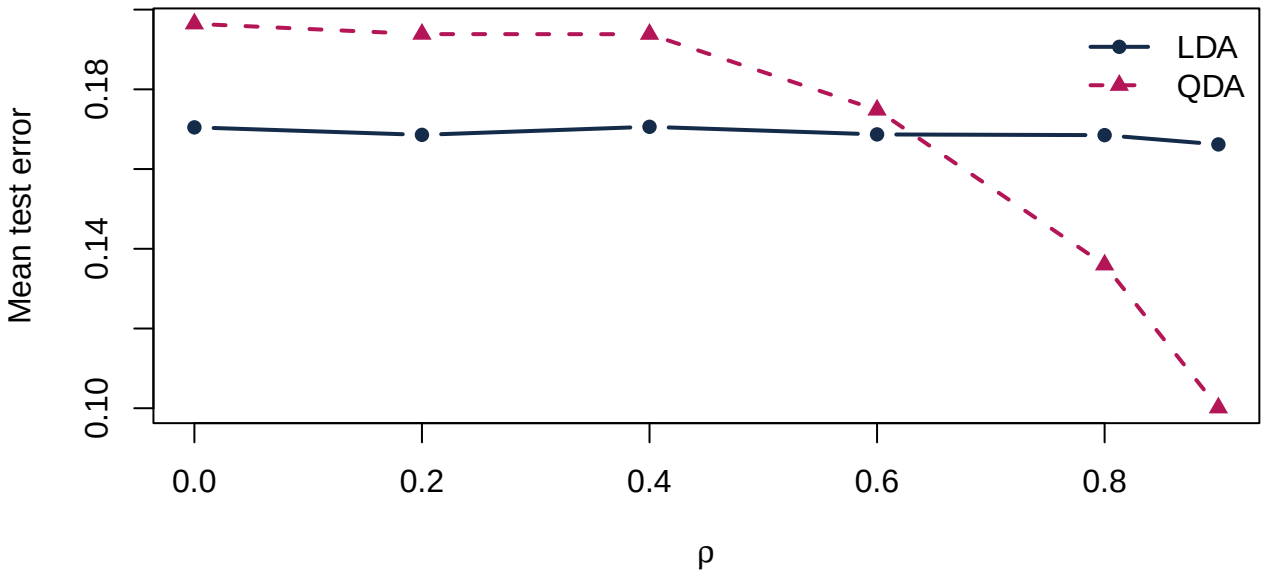
(b) Fifty repetitions per rho

```
averages <- t(sapply(rhos, function(rho) {
  rowMeans(replicate(50, simulate(rho)))
}))
knitr::kable(data.frame(rho = rhos, averages), digits = 4,
             caption = "Mean test misclassification rates over 50 repetitions")
```

Table 4: Mean test misclassification rates over 50 repetitions

rho	LDA	QDA
0.0	0.1705	0.1964
0.2	0.1686	0.1938
0.4	0.1706	0.1938
0.6	0.1687	0.1748
0.8	0.1685	0.1360
0.9	0.1662	0.1001

```
matplot(rhos, averages, type = "b", lwd = 2,
        pch = c(16, 17), lty = c(1, 2), col = c("#142B49", "#B31557"),
        xlab = expression(rho), ylab = "Mean test error")
legend("topright", c("LDA", "QDA"), lwd = 2,
       pch = c(16, 17), lty = c(1, 2),
       col = c("#142B49", "#B31557"), bty = "n")
```



(c) Explanation

LDA has lower average error at small ρ: the class covariances are similar, and pooling reduces estimation variance. At large ρ, QDA performs better by allowing different class covariances, reducing bias. This is the bias-variance trade-off. The eight noise variables add no class information but introduce estimation noise, especially for QDA's separate covariance matrices.

Question 4: Create and use an LDA/QDA analysis skill

(a) Define the skill

I created `lda-qda-stability/SKILL.md` to compare ordinary LDA and QDA using repeated cross-validation and a shuffled-label diagnostic. It is dataset-independent and requires a report of at most one printed page with a figure, table, and numerical results.

The skill is stored in this homework directory, outside automatically loaded skill folders. I supplied its path in an explicit request; it is not installed for automatic discovery.

The final skill file is **3725 bytes**, below the 5,000-byte limit.

(b) Apply the skill

I explicitly invoked the skill using the following prompt:

> Explicitly use homework/week-07/lda-qda-stability/SKILL.md to analyze homework/week-07/data/saheart.csv. The response is chd; use all nine other variables as numerical predictors, with famhist coded Absent=1 and Present=2. Use R and seed 43207. Seek the best predictive performance among the ordinary LDA and QDA candidates allowed by the skill. Reserve a stratified 20% test set first, compare models using shared repeated five-fold CV, and perform the training-only shuffled-label diagnostic. Review and freeze selection using training/CV results before evaluating the selected model once on the reserved test set.

> Save code and a one-page report in homework/week-07/q4-output. Append the report, full skill, byte count, and this prompt to the Question 4 submission.

The seed was **43207**. Both candidates used all nine predictors with the Question 1 coding. The fixed stratified split reserved 93 observations for testing and left 369 for training. I compared the candidates using the same stratified five-fold partitions, repeated ten times. All means, covariances, and empirical priors were estimated within each fold's training portion. No scaling, imputation, or embedding was needed.

For the placebo diagnostic, I shuffled only the training labels 50 times. Each shuffle repeated the same model-selection procedure, generating shared stratified folds for LDA and QDA and recording the lower mean CV error. This diagnostic was not used to tune the models.

(c) Review and submit

Before evaluating the test set, the agent checked the coding, shared folds, fitting-sample priors, and training-only placebo procedure. All model fits completed without rank failures. LDA had the lower mean CV error, so that selection was frozen. No skill revisions were needed. The training-only review is saved in `q4-output/training-review.txt`.

The agent then refitted LDA on all training observations and evaluated it once on the reserved test set. Subsequent report rendering reads saved results and does not refit or reevaluate the test set. The code blocks below are displayed for reproducibility rather than executed when this document renders.

The final one-page analysis report follows. The complete skill and analysis code are appended after it.

LDA or QDA?

SAheart | Prediction, stability & shuffled−label checks

Seed 43207 | 369 training / 93 test | 9 predictors

METHOD

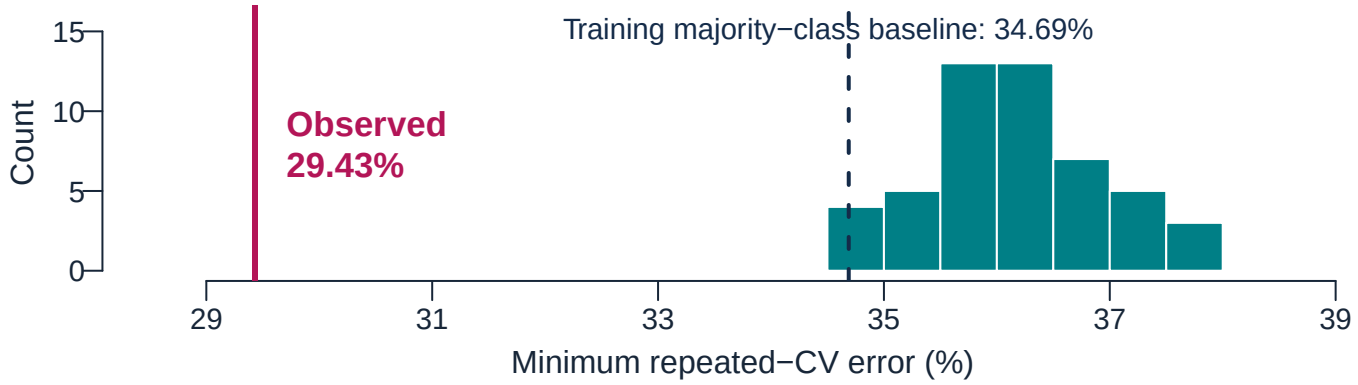
Predict CHD; famhist is coded Absent=1, Present=2. Compare ordinary LDA and QDA with shared stratified 5−fold CV, repeated 10 times. Each fit estimates its own priors, means and covariances; no learned preprocessing. Select the lowest mean error (ties: LDA). Reserve a stratified 20% test set before selection. Shuffle training labels 50 times and repeat the full CV comparison each time.

CROSS−VALIDATION

Model	Mean error	Split SD (pp)
LDA	**29.43%**	**0.63**
QDA	30.84%	1.22

Repeat wins: LDA 8 / QDA 1 / ties 1. Selected: LDA.

PERMUTATION CHECK



Shuffled−label mean: 36.11%. All 50 errors exceeded observed CV error.

TEST ERROR **17.20%**

16 / 93 errors; evaluated once

95% Wilson interval: 10.88%−26.13%

FINDINGS & LIMITATIONS

LDA had lower CV error and less variation across repeated splits; its shared covariance assumption may reduce estimation variance. Final fit: two 9−entry means, one 9 x 9 covariance; priors 0.653 / 0.347. The permutation comparison suggests predictive information, not causality. CV/test differences may reflect small−test sampling variability. Repeated CV folds overlap: split SD is not a confidence interval. Placebo is descriptive; clinical usefulness is unestablished.

Complete skill

```
---
name: lda-qda-stability
description: Explicitly invoked classification analysis using repeated cross-validation and a
 ↪  shuffled-label diagnostic for LDA and QDA.
---

# LDA/QDA stability analysis

Load this file only when the user explicitly supplies its path and asks to use it. Keep it
 ↪  outside automatically discovered skill folders.

Accept the dataset, response, predictors, coding, seed, and output location separately. Seek the
 ↪  lowest cross-validated misclassification rate among ordinary LDA and QDA; do not claim
 ↪  optimality beyond these candidates. Use empirical class priors estimated from each fitting
 ↪  sample. Report covariance rank failures rather than silently regularizing or dropping
 ↪  predictors.

## Analysis

1. Check predictor coding, finite values, class counts, and matrix dimensions. If data require
 ↪  imputation or other preprocessing, define a reproducible procedure fitted only on the
 ↪  fitting observations within each fold. Do not change the user's predictor coding.
2. Set and report a seed. Reserve approximately 20% of each class as a test set before model
 ↪  comparison. Keep the test observations out of all diagnostics, revisions, and selection
 ↪  until the final evaluation.
3. On the remaining training data, run stratified five-fold cross-validation repeated ten times.
 ↪  Use identical folds for both candidates. Fit each model from scratch in each fold, including
 ↪  any preprocessing and empirical priors. For each repetition, calculate the error over all
 ↪  out-of-fold predictions. Report each model's mean error and the standard deviation across
 ↪  repetitions; the latter describes split sensitivity, not a confidence interval.
4. Select the candidate with the lowest mean repeated-CV error. Resolve exact ties in favor of
 ↪  LDA. Report how often each model wins a repetition, counting ties separately. Do not select
 ↪  the seed or folds to obtain a preferred winner.
5. As an auxiliary placebo diagnostic, shuffle only the training labels 50 times, preserving
 ↪  class counts. For each shuffle, repeat the entire same CV procedure for both models and
 ↪  record the lower mean error. Generate stratified folds from those shuffled labels and share
 ↪  them between candidates. Compare the observed minimum CV error with this distribution and
 ↪  the training majority-class error. Do not describe chance error as necessarily 50%, use
 ↪  placebo results for tuning, or infer causality. Treat this as a descriptive diagnostic, not
 ↪  proof of predictive validity.
6. Review code and training/CV results, finalize instructions, and record the selected candidate
 ↪  before accessing test outcomes. Refit it on all training data and evaluate once on the
 ↪  reserved test set. Report test sample size, error count, misclassification rate, and a
 ↪  binomial Wilson 95% interval. Do not compare other models on the test set or revise after
 ↪  seeing its error. Subsequent report rendering must reuse saved results.

## Report

Produce a readable English report of at most one printed letter/A4 page, and verify the exported
 ↪  page count. Include the target, sample sizes, seed, candidate models, repeated-CV and
 ↪  placebo procedure, a small table of both candidates' CV mean errors and split standard
 ↪  deviations, and one figure showing the placebo minimum-error distribution with the observed
 ↪  minimum marked. Give the selected model, numerical final-test results, and a short
 ↪  interpretation. Explain that repeated folds overlap, permutation results are diagnostic, and
 ↪  a small reserved test set limits precision. Include numerical fitting details such as
 ↪  predictor count and pooled or class covariance dimensions. Avoid causal or clinical claims.
 ↪  Put the complete skill, byte count, invocation prompt, and reproducible code outside this
 ↪  one-page report.
```

Reproducible R analysis

The complete analysis code is reproduced below. It runs training/CV analysis and final test evaluation in separate stages; a saved review is required before the test stage. Existing final results prevent accidental test reevaluation. The dataset and file paths are supplied separately when invoking the skill.

```
library(MASS)
seed <- 43207
out <- "q4-output"
dir.create(out, showWarnings = FALSE)

folds <- function(y) {
  f <- integer(length(y))
  for (k in levels(y)) {
    ids <- which(y == k)
    f[ids] <- sample(rep(1:5, length.out = length(ids)))
  }
  f
}

cv_errors <- function(X, y, fold_sets) {
  t(sapply(fold_sets, function(f) {
    pred <- matrix(NA_character_, nrow(X), 2)
    for (j in 1:5) {
      fit_lda <- lda(X[f != j, , drop = FALSE], y[f != j])
      fit_qda <- qda(X[f != j, , drop = FALSE], y[f != j])
      pred[f == j, 1] <- as.character(predict(fit_lda, X[f == j, , drop = FALSE])$class)
      pred[f == j, 2] <- as.character(predict(fit_qda, X[f == j, , drop = FALSE])$class)
    }
    c(LDA = mean(pred[, 1] != y), QDA = mean(pred[, 2] != y))
  }))
}

train_analysis <- function() {
  if (file.exists(file.path(out, "final-results.rds"))) {
    stop("Final test results already exist; reuse them without revising selection.")
  }
  set.seed(seed)
  heart <- read.csv("data/saheart.csv")
  heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
  X <- data.matrix(heart[, names(heart) != "chd"])
  y <- factor(heart$chd)
  stopifnot(ncol(X) == 9, all(is.finite(X)), !anyNA(y))

  train_ids <- unlist(lapply(levels(y), function(k) {
    ids <- which(y == k)
    sample(ids, floor(0.8 * length(ids)))
  }))
  test_ids <- setdiff(seq_len(nrow(X)), train_ids)
  Xt <- X[train_ids, , drop = FALSE]
  yt <- y[train_ids]
  fold_sets <- replicate(10, folds(yt), simplify = FALSE)
  errors <- cv_errors(Xt, yt, fold_sets)
  avg <- colMeans(errors)
  selected <- names(avg)[which.min(avg)]
  wins <- c(LDA = sum(errors[, 1] < errors[, 2]),
```

```
            QDA = sum(errors[, 2] < errors[, 1]),
            Ties = sum(errors[, 1] == errors[, 2]))

  placebo <- replicate(50, {
    shuffled <- sample(yt)
    shuffled_folds <- replicate(10, folds(shuffled), simplify = FALSE)
    min(colMeans(cv_errors(Xt, shuffled, shuffled_folds)))
  })
  majority_error <- 1 - max(prop.table(table(yt)))
  summary <- data.frame(Model = names(avg), CV_error = avg,
                        Split_SD = apply(errors, 2, sd))
  write.csv(summary, file.path(out, "cv-summary.csv"), row.names = FALSE)
  write.csv(data.frame(errors), file.path(out, "cv-repetitions.csv"), row.names = FALSE)
  write.csv(data.frame(Error = placebo), file.path(out, "placebo-errors.csv"), row.names =
   ↪ FALSE)
  results <- list(seed = seed, X = X, y = y, train_ids = train_ids,
                  test_ids = test_ids, fold_sets = fold_sets, errors = errors,
                  summary = summary, selected = selected, wins = wins,
                  placebo = placebo, majority_error = majority_error)
  saveRDS(results, file.path(out, "training-results.rds"))
  print(summary)
  print(wins)
  cat("Selected:", selected, "\nTraining:", length(train_ids),
      " Test reserved:", length(test_ids), "\nMajority error:", majority_error,
      "\nPlacebo mean:", mean(placebo), "\n")
}

test_analysis <- function() {
  if (file.exists(file.path(out, "final-results.rds"))) {
    stop("Test was already evaluated. Render from saved results.")
  }
  r <- readRDS(file.path(out, "training-results.rds"))
  stopifnot(file.exists(file.path(out, "training-review.txt")))
  fit_fun <- if (r$selected == "LDA") lda else qda
  fit <- fit_fun(r$X[r$train_ids, , drop = FALSE], r$y[r$train_ids])
  pred <- predict(fit, r$X[r$test_ids, , drop = FALSE])$class
  r$confusion <- table(Observed = r$y[r$test_ids], Predicted = pred)
  r$test_errors <- sum(pred != r$y[r$test_ids])
  n <- length(r$test_ids)
  p <- r$test_errors / n
  z <- qnorm(0.975)
  center <- (p + z^2 / (2 * n)) / (1 + z^2 / n)
  half <- z * sqrt(p * (1 - p) / n + z^2 / (4 * n^2)) / (1 + z^2 / n)
  r$test_error <- p
  r$interval <- c(center - half, center + half)
  r$fit <- fit
  saveRDS(r, file.path(out, "final-results.rds"))
  print(r$confusion)
  cat("Selected:", r$selected, " Test error:", p,
      " Wilson interval:", r$interval, "\n")
}

args <- commandArgs(trailingOnly = TRUE)
if (length(args) && args[1] == "train") train_analysis()
if (length(args) && args[1] == "test") test_analysis()
```